Understanding and Implementing a Small-Scale Neural Radiance Field (NeRF)

Arpit Jain (202311016) Saransh Naik (202311077)
Param Sonawane (202311082)
Group: Synapse^3

Department of Computer Science and Engineering
Indian Institute of Information Technology Vadodara

Under the guidance of **Dr. Pratik Shah**

What is NeRF?

- NeRF stands for **Neural Radiance Field**
- It represents a 3D scene as a continuous function learned by a neural network (novel view synthesis)
- Input: 2D images with known camera poses
  - Every 2D image: a 5D coordinate - location (x, y, z) and view direction (θ, ϕ)
- Output: A model that can render the scene from any viewpoint in 3D space
  - Representation: colour (r, g, b) and volume density (σ)
  - $F_\Theta : (x, d) \rightarrow (c, \sigma)$

Why NeRF?

- Traditional 3D reconstruction methods are limited in detail
- NeRF achieves photo-realistic novel view synthesis
- Applications: **Virtual Reality, Robotics, AR, Gaming**

Dataset and Input Format

- The dataset contains 60 RGB images (each 100x100 px in size) of each object and a corresponding `transforms.json` file
- `camera_angle_x` gives the horizontal field of view (in radians); this is common for all frames
- Each frame contains:
  - `file_path`: image name
  - `transform_matrix`: a 4x4 matrix giving camera pose in world (c2w pose) coordinates - [R t; 0 1]
    - R: 3x3 rotation matrix - camera orientation [Right Up Forward]
    - t: 3x1 translation vector - camera position - [x; y; z]
    - Last row: homogeneous coordinates (same for every frame)

Camera Model and Ray Generation

- Each pixel generates a ray from the camera center
- Ray = origin + depth × direction or $R = o + t \times d$ (parametric equation of a line in space)
- Direction depends on camera intrinsics (H, W, focal) and pose:

Key Function:

`get_rays(H, W, focal, pose)` returns ray origins and directions, where H and W are height and width of image (100 each), pose is obtained from transform matrix, and focal length is calculated as:

$$f_x = f_y = \frac{W}{2\tan\left(\frac{\theta_x}{2}\right)}$$

Network Architecture

- 8-layer (fully connected) MLP (MultiLayer Perceptron), 256 units (neurons) per layer
- Skip connection every 4 layers - original input concatenated to preserve low-frequency structure and prevent network from forgetting input coordinates (training stabilization)
- Output: **RGB (3 channels: r, g, b) + Density (1 channel: σ)**
- Activation: ReLU (Rectified Linear Unit) in last layer to introduce non-linearity - outputs input if positive, 0 otherwise

Input Features

3D point + Positional Encoding of frequencies

Positional Encoding

- Having the network directly operate on $xyz\theta\phi$ input coordinates results in renderings that perform poorly at representing high-frequency variation in colour and geometry
- Deep networks are biased toward learning lower frequency functions (Rahaman et al.)
- Positional encoding allows network to represent high-frequency details (like fine texture and sharp detail)
- Each coordinate p is mapped as the following encoding function:

$$\gamma(p) = [\sin(2^0\pi p), \cos(2^0\pi p), \ldots, \sin(2^{L-1}\pi p), \cos(2^{L-1}\pi p)]$$

- where γ is a mapping from $\mathbb{R}$ to $\mathbb{R}^{2L}$
- Thereafter: $F_\Theta = F'_\Theta \times \gamma$, where F'_Θ is a regular MLP
- L is the number of frequency bands; $L = 10$ in our implementation

Volume Rendering Principle

- Used to predict colour and density at all sampled 3D points along a ray
- Alpha compositing: how much a point contributes before light gets blocked (opacity); $\alpha_i = 1 - e^{-\sigma_i \delta_i}$
- Combine predicted values using alpha compositing:

$$C(r) = \sum_{i=1}^{N} T_i (1 - e^{-\sigma_i \delta_i}) c_i$$

- where
  - T_i is transmittance (how much light is preserved from point 1 to point i); $T_i = e^{-\sum_{i=1}^{N} \sigma_i \delta_i}$
  - c_i is the (r, g, b) colour value of ith point
  - σ_i is density at i^{th} point
  - $\delta_i = t_{i+1} - t_i$ is distance between adjacent points
- $C(r)$ is the final pixel colour of a frame for ray r

Training Setup

- 8 layers, 256 units per layer, $L = 10$ frequency bands
- Optimizer: Adam (learning rate = 5e-4) - helps stabilize training
  - Adjusts learning rates per parameter: handles uneven gradients from positional encoding
  - Momentum smoothing: stabilizes noisy gradients from ray sampling
  - Fast and stable convergence: learns a good radiance field in very few steps
- Loss: Mean Squared Error (MSE): $\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(C_{\text{pred},i} - C_{\text{g.t.},i})^2$
  - C_{pred} is predicted RGB value by model
  - $C_{\text{g.t.}}$ is ground truth (actual) RGB value from dataset
- Iterations: 2000
- Samples (divisions / points) per ray: 64
- Validation every 25 iterations

Evaluation Metrics

- **PSNR:** Peak Signal to Noise Ratio - calculates pixel level accuracy; $\text{PSNR} = 20\log_{10}(\frac{1.0}{\sqrt{\text{MSE}}})$
- **SSIM:** Structural Similarity Index Measure - perceptual structure based on edges, textures, patterns, luminance, and contrast consistency
- **LPIPS:** Learned Perceptual Image Patch Similarity - uses pretrained networks (AlexNet/VGG) to measure realism, perceptual distance, high-level differences (how humans judge image quality)

Goal:

Higher PSNR, higher SSIM, lower LPIPS = better reconstruction.

Results and Observations

- Steady improvement in PSNR, SSIM, and LPIPS over training
- Large quality jump in the first 1000 iterations, then slower gains
- Final reconstruction shows good structural consistency and perceptual quality

Metric Progression

Iteration	PSNR	SSIM	LPIPS
0	7.37	0.029	1.0115
1000	22.31	0.774	0.2275
2000	24.57	0.863	0.1644

- Results in original NeRF:

PSNR	SSIM	LPIPS
40.15	0.991	0.023

PSNR Graph

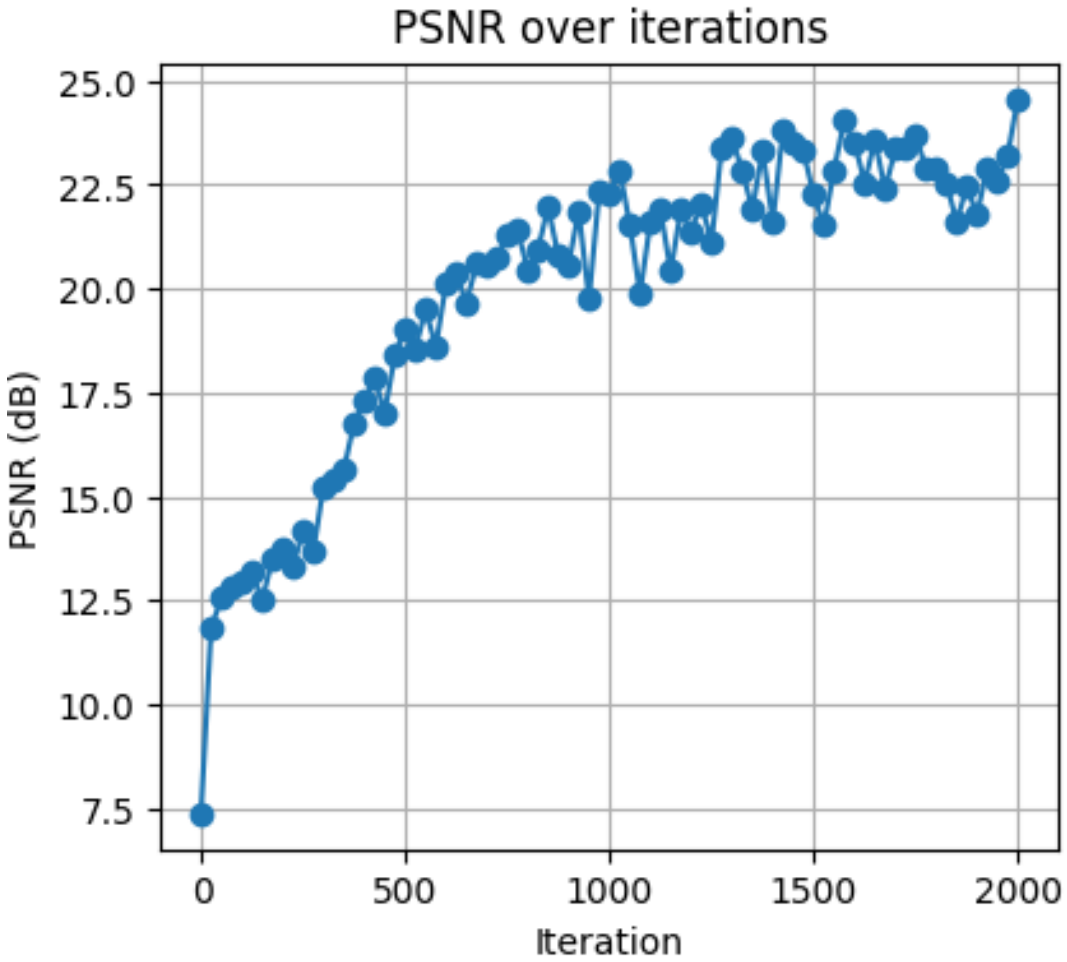

Figure: PSNR Graph after 2000 iterations

Conclusions and Future Work

- Implemented a small-scale NeRF training pipeline
- Successfully generated 3D views and evaluated image metrics
- Issues observed: scale mismatch and ray normalization.
- Future improvements:
  - Hierarchical sampling
  - Coarse-to-fine NeRF
  - Better dataset preprocessing

Reference

[1] Mildenhall, B., Srinivasan, P. P., Tancik, M., Barron, J. T., Ramamoorthi, R., & Ng, R. *NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis*. Communications of the ACM (2021). **doi:** 10.1145/3503250